

Market-anchored boosted multinomial probit

Does training a nonlinear tree ensemble directly under the multinomial-probit race likelihood improve on the existing probit links and the production softmax booster?

Branch `research/boosted-probit`

1 September 2026

What this document is. The detailed research report for the boosted-probit experiment. The repository-standard model report is rendered separately from this run's own artifacts by `hkjc build-report --model probit_offset_boosted`, through the same bundle and publication path as the four existing models; it says what the model scored. This document says how it was derived, validated, profiled, and three times nearly mis-concluded — the third time in this report's own statistics rather than in the model.

Scope. The branch is not merged. The production default remains `configs/models/softmax_offset_production.json`, and the new model carries `standard` rather than `production` status in the publication config.

The answer, up front. No. Direct probit training beats both baselines on the mean, but neither margin survives a test across test years: $t = -1.40$ against the production booster and $t = -0.46$ against the link-only probit, on 6/11 years favourable. Essentially the whole gain belongs to the probit *link*, which the repository already has and which costs no fitting at all. The recommendation is to keep the softmax booster.

What it does not establish. No profitability claim: every price is a result-page price recorded after the pool closed, and no return is computed. The utility scale is fitted on earlier *test* years, which are development data under the repository's standing limitation, so this is not a locked prospective result.

Contents

1	Executive summary	3
2	Motivation	4
3	Existing architecture, reproduced	4
4	Multinomial probit mathematics	4
5	The market anchor	5
6	Probability engine	5
7	Gradient derivation	5
8	Boosting algorithm	6
9	Optimisation	7
10	The utility scale, and two failures	7

11 Results: the comparison ladder	8
12 Interaction analysis	10
13 Calibration and sharpness	11
14 Longshots	11
15 Runtime and engineering tradeoffs	12
16 Tail and variance experiments	12
17 Limitations	12
18 Recommendation	13

1 Executive summary

The plan’s §45 asks eight questions. Answering them directly:

1. **Did direct boosted probit beat the current production model?** On the mean, yes, by -0.001352 — but **not distinguishably**: $t = -1.40$ across 11 test years, favourable in only 6 of them. That is 34.2% of the production edge on the mean and nothing that clears a significance bar.
2. **Did it beat the existing link-only probit?** **No.** -0.000410 at $t = -0.46$ across years — indistinguishable from zero. This is the comparison the plan’s §46 calls the key one, and it is the answer that decides the experiment: the link-only `probit_offset` already collects -0.000844 against the production booster *for no fitting at all*, and direct training adds nothing measurable on top of it.
3. **Was the gain statistically stable?** **Across seeds yes, across years no**, and the distinction is the whole result. 5/5 seeds favourable at $t = -16.03$, but only 6/11 years at $t = -1.40$. Five seeds score the *same races*, so their agreement measures seed stability, not generalisation; test years are the independent replicates. See §11.
4. **Was it worth the computational cost?** Training is about $30\times$ the softmax booster’s — 100 minutes against 3 for a twelve-year five-seed sweep. Scoring is free. See §15.
5. **Did the probit objective learn different structure?** The mean function it learns is measurably different, not a rescaling: scored through the *same* softmax link its margins differ from the Offset’s by -0.000610 . See §12.
6. **Is the market anchor numerically robust?** Yes: the ratio anchor returns q to 1.1×10^{-16} at zero correction, at every scale, and the gradient module’s probabilities are bit-identical to the integrator’s.
7. **Is the implementation production-feasible?** Yes as a model; the open question is rebuild cadence rather than correctness.
8. **Did it beat post-hoc probit?** See question 2 — that is the same comparison, and it is the one the plan’s §46 calls the key one.

The two things this experiment nearly got wrong

An unstable utility scale produced a spectacular false positive. The plan’s alternating scale fit gave -0.016 against the market at 100 rounds and $+0.004$ at 200. The first number is 3.5 times the production edge and would have been reported as a result had the second not been measured. §10.

Matched nominal hyperparameters are not matched capacity. The probit gradient carries $1/\sigma$ and its Gauss–Newton Hessian $1/\sigma^2$, so the leaf weight scales like σ and the effective step at $\sigma \approx 1.8$ is about twice the softmax objective’s. At the production learning rate the boosted probit **loses to the market**; on the plateau from 0.002 to 0.003 it beats both baselines with a *smaller* correction. A negative result was reported internally before this was found, and it was wrong. §10.

And an across-seed standard error nearly manufactured a significant result. The first version of this report’s generator computed significance across the five seeds, which gave the headline $t = -16.0$. Five seeds score the *same races*: their deltas are five measurements of one sample, and their tight agreement measures determinism, not generalisation. Recomputed across test years — the actual independent replicates — the same number is $t = -1.4$, and the comparison against

the link-only probit is $t = -0.5$. The recommendation logic had also been written to test only the *sign* of that second comparison, so the two faults compounded: the report would have filed a null as promising. Both are fixed, and the conclusion reversed when they were.

2 Motivation

The production model is a market-anchored softmax booster: the market’s normalised implied probability enters XGBoost as a fixed additive base margin $\log q$ and the booster learns a per-runner correction against a racewise winner-softmax likelihood. At zero correction the model *is* the market exactly, because $\text{softmax}(\log q) = q$.

A softmax choice model is the Gumbel case of a random-utility argmax. The repository’s Phase 6 work replaced those Gumbel shocks with normal ones and found the normal link better on both of its scoreboards: -0.012557 standalone and -0.000943 market-relative, every year and every seed favourable. But both results are **link-only** — the mean utilities came from a softmax fit, and `findings.md` says so explicitly: *”no native joint probit likelihood was trained, and metadata records `mean_training=softmax_surrogate` for that reason.”*

That leaves the obvious question. A linear probit mean is restrictive and trees fix that; but if the probit *likelihood* is the better one, the ensemble should be fitted under it rather than lifted from a fit under a different one. This experiment fits one.

3 Existing architecture, reproduced

Nothing was implemented until the recorded results were reproduced from the committed artifacts, on `is_honest_scale` rows (2015 has no earlier season to fit a scale on and is excluded from every headline in this document, as it is from the published ones).

Quantity	Reproduced	<code>findings.md</code>
market race log loss	2.028805	—
Offset, softmax link on its margins	2.025390	—
<code>probit_offset</code>	2.024447	—
Offset — market	-0.003415	—
<code>probit_offset</code> — market	-0.004358	—
<code>probit_offset</code> — Offset	-0.000943	-0.000943

Exact to the digit. One note carried through the whole document: `findings.md` quotes the production edge as -0.004647 , which is the *calibrated* figure. The raw-softmax-on-margins measurement used consistently here is -0.003957 on the shared-race population. The two are different measurements of the same model, and mixing them would move a candidate by 0.001 in whichever direction flattered it.

4 Multinomial probit mathematics

For runners $1, \dots, n$ with mean utilities μ_i , iid normal shocks and scale σ , the winner probability collapses to a one-dimensional integral over the winner’s own shock:

$$P(i \text{ wins}) = \mathbb{E}_z \left[\prod_{j \neq i} \Phi \left(\frac{\mu_i - \mu_j}{\sigma} + z \right) \right], \quad z \sim N(0, 1).$$

That collapse is what makes an exact answer available at all, and it is why the repository’s integrator is Gauss–Hermite quadrature rather than Monte Carlo. The existing module computes it as a log-sum so a product of 13 normal CDFs cannot underflow, dividing out the $j = i$ factor in log space so the inner array stays a dense matrix operation. None of that was changed.

For two runners there is a closed form, $P(1 > 2) = \Phi((\mu_1 - \mu_2)/(\sqrt{2}\sigma))$, which the integrator matches to 1.1×10^{-16} .

5 The market anchor

Under a softmax link, $\text{softmax}(\log q) = q$ exactly. Under probit it does not: $C_{\text{probit}}(\log q, \Sigma) \neq q$, and on real margins the discrepancy reaches 0.20. Feeding $\log q$ to a probit link therefore *moves the baseline*, and the unanchored diagnostic loses to the incumbent by four times the model’s whole edge — entirely from a baseline shift that would have been attributed to the link.

The repository’s answer is a ratio anchor, and it is exact:

$$p = \text{normalize}_r \left(\frac{q \cdot P(\log q + c)}{P(\log q)} \right),$$

which returns q when $c = 0$ because the numerator and denominator are then the same numbers. Measured at 1.1×10^{-16} , at every utility scale — which is also why σ is *unidentified* before the first tree: the loss is exactly flat in it while the correction is zero.

The boosted architecture uses this anchor unchanged, with $c = f_\theta(x)$.

6 Probability engine

Reused, not rebuilt. §5 of the plan asks for a new engine only if the existing one cannot supply gradients, batch, or stay stable; it can do all three. The one addition is a probabilities-only entry point that calls the integrator’s own `_race_stack_probabilities`, because the boosting loop wants probabilities without the gradient’s λ array once per round.

§24: how many nodes does the *gradient* need? Measured on a real 2024 training slice (130,629 rows, 10,496 races) against a 128-node reference:

nodes	max abs prob error	max rel gradient error	race log-loss error	seconds
8	3.2e-03	1.4e-01	2.4e-05	0.16
16	1.8e-04	1.2e-02	3.0e-06	0.27
24	1.6e-05	1.3e-03	2.4e-07	0.39
32	2.2e-06	1.5e-04	2.5e-08	0.51
48	3.2e-08	3.3e-06	6.7e-10	0.75
64	6.5e-10	8.5e-08	1.4e-12	0.97

32 nodes for training, 64 for scoring. At 32 the induced race-log-loss error is 2.5×10^{-8} against an effect of 9.4×10^{-4} — **38,000× smaller**.

7 Gradient derivation

The plan’s derivation is for a different model, and using it would have trained the wrong objective. §12 derives $\partial(-\log P_w)/\partial\mu$ for the *unanchored* probit, where only the winner’s

probability appears and one row of the Jacobian suffices. The prediction here is anchored. Writing

$$s_j = \log q_j + \log P_j(\mu) - \log P_j(\log q)$$

makes the structure plain: $p = \text{softmax}_r(s)$, so $\partial L / \partial s_j = p_j - y_j$ — exactly the production objective’s residual — and

$$\boxed{\frac{\partial L}{\partial \mu_i} = \sum_j (p_j - y_j) \frac{\partial \log P_j}{\partial \mu_i}}$$

The anchor’s denominator does not depend on μ and drops out. What does not drop out is that every runner’s s depends on every runner’s μ , so this needs the **full** $n \times n$ **Jacobian**.

With $a_{jk} = (\mu_j - \mu_k) / \sigma + z$ and $\lambda(a) = \phi(a) / \Phi(a)$:

$$\frac{\partial P_j}{\partial \mu_j} = \frac{1}{\sigma} \mathbb{E}_z \left[\prod_{k \neq j} \Phi(a_{jk}) \sum_{k \neq j} \lambda(a_{jk}) \right], \quad \frac{\partial P_j}{\partial \mu_i} = -\frac{1}{\sigma} \mathbb{E}_z \left[\prod_{k \neq j} \Phi(a_{jk}) \lambda(a_{ji}) \right].$$

Both share the product term the probability already forms, and λ is the same (races, nodes, n, n) shape as the log-CDF array — so the gradient costs about one more probability evaluation rather than n of them. Only the vector–Jacobian product is formed; the Jacobian is contracted inside the node loop and never materialised.

Curvature. The true race Hessian is dense. XGBoost consumes one scalar per row, so the diagonal used is Gauss–Newton, $h_i = J_{ii}^2 p_i(1 - p_i)$, nonnegative by construction rather than by clipping and reducing to the production softmax’s $p(1 - p)$ when the link is the identity.

§13’s mandatory validation

Check	Result	Tolerance
probabilities vs quadrature_probit_probabilities	0.0 (bit-identical)	—
anchor identity at zero correction	1.1e-16	1e-12
gradient vs central difference, worst of 48 cells	5.6e-9 relative	1e-6
curvature nonnegative	yes	—
sum of probabilities per race	1 to 2e-12	1e-9

48 cells: field sizes $\{5, 8, 12, 14\} \times$ regimes {favourite-heavy, near-uniform, longshot-heavy} $\times$ steps $\{10^{-3}, 10^{-4}, 10^{-5}, 10^{-6}\}$. Best step 10^{-4} ; 10^{-6} degrades to $\sim 1\text{e-}9$ from cancellation, as expected.

One bug worth recording, because it looked numerical and was not. The first version failed its own check at 0.8000000000006817. That is 4/5: the analytic gradient differentiates the race-loss *sum* (the convention `softmax_obj` uses, since XGBoost sums rows) while the finite-difference helper differenced the *mean*, and the fixture had five races.

8 Boosting algorithm

An XGBoost custom objective on the production training path, not a bespoke booster. §14 offers a hand-rolled first-order loop as a reference and says not to build one merely because the plan suggests it — and the repository already trains its production model through `xgb.train(..., obj=softmax_obj)` with `base_margin=log q`, which is structurally the same problem. Reusing that path means the two architectures differ in exactly one argument, and the boosted probit inherits the production model’s regularisation, column sampling, histogram method, determinism

and threading. A parallel booster would have turned §22’s capacity comparison into a comparison of two training stacks.

Per round: form μ from the base margin plus the ensemble so far, integrate probabilities, form the residual $p - y$, take one more pass for the gradient and the curvature together, hand XGBoost the two vectors.

9 Optimisation

Profiled before optimised, as §23 asks. Two bottlenecks, both fixed.

Three integration passes per round became two. The objective computed probabilities, then the gradient, then the curvature. The gradient and curvature cotangents now share a pass (the VJP accepts a stack of cotangents), and the probabilities-only path stops building a λ array it discards. **62% of a round $\rightarrow$ 44%.**

Threading. `log_ndtr` releases the GIL and the integrator already had a thread knob; the gradient inherits it.

threads	nodes	probability	gradient	per round	400 rounds
1	32	0.86s	1.38s	2.24s	~15 min
1	64	1.63s	2.66s	4.29s	~29 min
8	32	0.14s	0.22s	0.36s	~2 min
8	64	0.27s	0.43s	0.70s	~5 min

So direct probit training is computationally feasible: a 400-round fit over a decade of races is two minutes. Cells are therefore fitted *serially* — each already saturates the machine through the integrator’s threads, and fanning cells across processes would divide the same cores twice.

10 The utility scale, and two failures

The utility scale is the single most dangerous parameter in this architecture, and the repository already said so: *“the utility scale is the whole game, and theory gets it wrong.”* It failed twice here, in two different ways, and both failures were caught by controls rather than by inspection.

Why it is unidentified at the start

With $f = 0$ the anchor returns q exactly for *any* σ , so the training loss is exactly flat in it. A fit before the first tree is meaningless. It becomes identified as soon as $f \neq 0$, because σ divides the market’s own spread as well as the correction’s.

Failure 1: the plan’s Option C

§18 Option C is to alternate — boost, refit σ on the training window, continue. Test year 2024, seed 7, refitting every 64 rounds:

rounds	fitted σ	boosted probit – market
100	0.66	−0.016128
200	0.41	+0.004470

At 100 rounds it appeared to beat the market by three and a half times the production edge. At 200 rounds it lost to the market. σ was running to its lower bound.

The cause is systematic rather than random: refitting on the *training* loss selects a small scale, because the training correction is overfit and dividing by a small scale sharpens it. Sweeping σ on the softmax booster’s own margins shows it from the other side — the test-optimal scale *rises* as the mean function grows while the training-optimal scale falls:

σ	on 100-round margins	on 400-round margins
0.66	−0.007820	+0.008432
1.00	−0.007288	−0.009001
1.50	−0.005326	−0.010168
2.00	−0.004144	−0.008923

An alternation that cannot see held-out data chases the wrong one, and its fixed point is a bound. §10’s recalibration control also rules out the simple explanation: amplifying the softmax correction by a factor fitted on training data ran the factor to the grid ceiling and bought only −0.000466, so the −0.016 was not a rescaling — it was a lucky point on an unstable trajectory.

Failure 2: the same error one level out

A development sweep then fitted the *scoring* scale on the previous year’s margins from the same model. But that model was trained on years including the previous one, so those margins are in-sample, and the scale fit again ran to 0.30–0.45 and the model lost to the market by +0.006 to +0.018.

What the architecture uses

`scale_mode="fixed"`: trees train at a declared scale and the *scoring* scale is fitted forward per test year by the stage, on strictly earlier years — the identical procedure `probit_offset` gets. That is what makes the comparison a comparison of likelihoods rather than of scale-selection procedures, and it removes a feedback loop whose fixed point is a bound. `scale_mode="alternating"` is retained so the failure reproduces rather than only being described.

The canonical run’s forward-fitted scoring scales land at **1.73–1.80**, interior to the (0.25, 12.0) bounds and near the 1.99–2.23 the link-only fits produce. No year hit a bound.

11 Results: the comparison ladder

Canonical scale: 11 test years $\times$ 5 seeds $\times$ 400 rounds, at the validated learning rate 0.0025. Every arm is scored on the *same* race keys, and each paired delta is computed per race and then averaged, never as a difference of two separately aggregated populations.

§21’s ladder

Arm	race log loss	Δ vs market
F. boosted probit (directly trained)	2.023757	−0.004905
C/D. link-only <code>probit_offset</code>	2.023818	−0.004844
E. softmax link on boosted-probit margins	2.024052	−0.004610
B. production Offset (softmax booster)	2.024662	−0.004000
A. market	2.028662	—

§28’s paired comparisons

Comparison	mean Δ	se	t	seeds better	races better
boosted probit vs production Offset	−0.000905	0.000056	−16.03	5/5	0.502
boosted probit vs link-only probit_offset	−0.000061	0.000050	−1.24	4/5	0.504
<i>link-only vs production</i> (baseline, reproduced)	−0.000844	0.000022	−37.97	5/5	0.527
probit link vs softmax link, <i>on boosted margins</i>	−0.000295	0.000003	−87.84	5/5	0.575
boosted margins vs Offset margins, <i>same softmax link</i>	−0.000610	0.000055	−11.15	5/5	0.500

Those standard errors are across seeds, and they are not a generalisation test. An earlier version of this section called them *the conservative choice*. They are not. Five seeds score the *same races*, so five deltas are five measurements of one sample; their agreement measures how deterministic the training is, which is worth knowing and is not evidence that the effect recurs. The independent replicates are test years.

The same two comparisons, across test years

Comparison	mean Δ	se (years)	t (years)	years better	t (seeds)
boosted probit vs production Offset	−0.001352	0.000968	−1.40	6/11	−16.03
boosted probit vs link-only probit_offset	−0.000410	0.000900	−0.46	6/11	−1.24

The last two columns are the finding. $t = -16.0$ and $t = -1.4$ are the same measurement of the same model under two different notions of what counts as an independent observation, and only the second one answers *will this hold next season*.

Headline. On the mean, boosted probit beats the production softmax booster by **−0.001352** — 34.2% of the production model’s entire edge over the market. But across test years that is $t = -1.40$ on 6/11 years favourable, and against the link-only probit — the baseline that costs no fitting — it is **−0.000410** at $t = -0.46$. **Neither margin survives a year-level test.**

Per year

Test year	vs production Offset	vs link-only probit
2016	+0.002262	+0.003389
2017	−0.000576	−0.000093
2018	+0.001022	+0.002164
2019	−0.001767	−0.001718
2020	+0.000041	+0.001406
2021	+0.000757	+0.001287
2022	−0.005591	−0.003928
2023	−0.001198	−0.000505
2024	−0.003653	−0.003570
2025	+0.001812	+0.002735
2026	−0.007985	−0.005676

Reconciling this table with the published model report

`results/reports/probit_offset_boosted/report.html` reports the boosted architecture at a race log loss of 2.023697 where the ladder above says 2.023757. Both are right and the gap is not

rounding: the published report scores the *bagged* ensemble — one probability per runner, averaged over the five seeds before the loss is taken — while this ladder averages each seed’s own loss so that every arm is scored the way its comparator is. Jensen’s inequality makes the bagged figure the smaller of the two, necessarily and by 6×10^{-5} here. Neither is used to compare against the other document’s arms, which is the only way the difference could mislead.

Forward-fitted scoring scales span 1.657 to 1.741, interior to the (0.25, 12.0) bounds. `scale_on_bound_years:` `[]`.

12 Interaction analysis

What the diagnostic can and cannot show, and what was actually run. Split frequency and gain say which features an ensemble used; only the count of feature pairs on a shared root-to-leaf path says an *interaction* was learned, because two features with large separate gains that never share a path are being used additively. That pair-count diagnostic was specified and *was not run*: no gain, split-share or pair-count table appears in this report, and no committed artifact holds one. The earlier version of this paragraph named a module as computing both, which was a statement about code that existed rather than about evidence that was produced, and the module has since been removed as never having been called. What §20 rests on here is the depth ablation below, which is real and reproducible from `depth_ladder.csv`, together with §11’s identical-link comparison.

§35’s depth ablation: the motivating hypothesis, tested directly

If the probit objective’s advantage is about representing nonlinear interactions, it must be *absent* at depth 1, where a tree ensemble is additive by construction and has no tree interactions at all. Honest validation year inside the training window ($\text{fit} \leq 2022$, $\text{validate } 2023$), each objective at its own tuned learning rate, σ fixed at 1.8:

Max depth	softmax Δ vs market	probit Δ vs market	probit – softmax
1 (additive)	−0.004279	−0.001614	+0.002665
2	−0.003922	−0.002806	+0.001115
3	−0.003470	−0.003552	−0.000082
4	−0.003580	−0.003927	−0.000347
5 (production)	−0.003646	−0.004269	−0.000623
6	−0.002793	−0.003608	−0.000814

Monotone across all six depths, and it crosses zero at depth 3. At depth 1 the probit objective is *much worse* than softmax — +0.00267, nearly three times the effect the whole experiment is chasing. The gap closes at every step and the probit objective ends up −0.00081 ahead at depth 6.

This is as direct a confirmation of the plan’s motivating hypothesis as the data can give: **the probit likelihood’s advantage is conditional on the mean function being able to represent interactions, and without that capacity the likelihood is a liability.** It also explains why the repository’s linear-probit intuition was right to be suspicious of a linear mean, and why the link-only experiments – which inherit a depth-5 softmax mean – saw a gain at all.

One incidental observation worth recording because it complicates the reading: the *softmax* objective is best at depth 1 on this validation year (−0.004279 against −0.003646 at the production depth 5). So the production depth is not optimal for the softmax objective on this year either,

and part of what the depth ladder shows is the two objectives having different optimal capacities rather than one being uniformly better.

The stronger and simpler evidence that the mean function differs is in §11’s ladder, and it needs no tree diagnostic at all. Scoring the boosted margins and the Offset margins through the *identical* softmax link isolates the mean function from the link entirely, and the two differ. A pure rescaling would not survive that comparison, and §10’s recalibration control tested rescaling directly and rejected it: amplifying the softmax correction by a factor fitted on training data ran to the grid ceiling and recovered almost none of the difference.

The correction is smaller, not larger. At the tuned learning rate the probit-trained correction has standard deviation 0.102 against the softmax booster’s 0.145. Whatever the probit objective is doing, it is not winning by being more aggressive — which is the opposite of what a capacity advantage would look like, and it is why §22’s capacity discussion in this experiment runs the other way from the plan’s expectation.

What was not done. SHAP interaction values were not computed. §20 permits them “if computationally feasible” and warns against interpretability theatre; the pair-count diagnostic answers the question the plan actually poses — *does the probit objective induce a meaningfully different partitioning of feature space* — and a SHAP interaction matrix over 28 features and 400 trees would have added cost without adding an answer. The depth ablation §35 asks for is reported at development scale only, for the reason §17 gives: canonical compute goes to a small number of serious candidates, and a canonical depth sweep is six times the canonical cost.

13 Calibration and sharpness

Race-level scores are what the artifacts persist, so calibration and sharpness are reported from the published model report rather than recomputed here: [results/reports/probit_offset_boosted/report.html](#) carries the reliability curves and the yearly reliability table that the four existing model reports carry, rendered by the same sections.

The one thing worth stating in this document is the principle the plan insists on and the published report also honours: a flatter model is easier to calibrate and no better, so calibration RMSE is never shown without sharpness beside it. The boosted architecture’s corrections are *smaller* than the incumbent’s at the tuned learning rate (sd 0.102 against 0.145), which means any advantage it has is not bought with extra confidence.

14 Longshots

The plan’s §33 per-bucket table was not produced for this experiment, and saying so is better than the previous version of this paragraph, which named a module as computing buckets this section never shows. What exists over the same population is the market-probability reliability band table in the published model report, [results/reports/probit_offset_boosted/report.html](#), in the same form the other four model reports carry it. Had the bucket table been built, the quantity worth reporting per bucket would be its *share of winners* and its log-loss *contribution* rather than a per-runner loss — a bucket holding 2% of winners cannot matter more than 2% however badly it is predicted.

The reason this matters more for probit than for softmax is numerical, and it is settled rather than open. A winner-count QMC estimator cannot resolve a probability below $1/m$; at the plan’s suggested 2,048 draws that floor is 4.9×10^{-4} , and real longshots here sit at 10^{-3} to 10^{-4} — exactly the range race log loss is dominated by. The repository already replaced QMC with Gauss–Hermite quadrature for that reason, and this experiment inherits it: the integrator resolves 1.1×10^{-9} and

matches the analytic binary result to 1.1×10^{-16} , so no longshot in this data is near the engine’s resolution.

15 Runtime and engineering tradeoffs

Model	Rounds	Train time per cell	Score time	Notes
production Offset (softmax)	400	~3.4s	<0.1s	one threaded tree fit
link-only <code>probit_offset</code>	—	0 (no fit)	0.50s/year	consumes existing margins
boosted probit	400	58–155s	0.04s	44% of the fit is the objective

Cell time grows with the training window, from 58s for the 2015 cell to about 155s for 2026, so a twelve-year five-seed sweep is roughly 100 minutes against about three minutes for the softmax equivalent — a factor of ~30. The scoring cost is negligible either way: the integrator does a test year in 0.04s and the whole scale-fitting and calibration pass takes 8.6 seconds.

Where the time goes, measured. The objective is 44% of a round after fusing three integration passes into two; the remainder is XGBoost’s own tree construction. The objective itself is dominated by one `ufunc`, `scipy.special.log_ndtr`, over a `(races, nodes, n, n)` array, which is why threading buys 10–16× and why further gains would have to come from the array’s shape rather than from the language.

Is 30× affordable? For research, plainly yes. For production the honest answer is that it depends on cadence: a full twelve-year rebuild moves from minutes to hours, which is tolerable weekly and not tolerable interactively. Nothing about the architecture requires the whole history to be refitted for a single new race day.

16 Tail and variance experiments

Not run, and the plan says not to start with them (§37, §38). The homoskedastic $\epsilon \sim N(0, \sigma^2)$ model is what this experiment tests, and the repository’s prior work already found that structured covariance did not help and that more elaborate shock distributions did not improve the standalone generative model after scale fitting. Student-*t* tails and a boosted variance model are named here so their absence is a decision rather than an omission.

17 Limitations

The learning rate was selected on one validation year. The sweep in §10 used 2023 as an honest held-out year inside the training window. One year is a thin basis, and the plateau from 0.002 to 0.005 is broad; 0.0025 was chosen inside it rather than at the single-year argmax of 0.003 for exactly that reason. A candidate whose advantage depends on a hyperparameter selected on one year should be treated as provisional however good its canonical number.

The scale is fitted on earlier test years. That is the same limitation the published `probit_offset` report carries, and it is inherited rather than introduced: those years are development data under the repository’s standing limitation, so this is not a locked prospective result.

No window is genuinely unseen. Every season in the archive has been development data, here and in the reference project. The only real confirmation is a prospective test of a locked model against timestamped pre-race prices.

The curvature is an approximation and is not the only one available. The Gauss–Newton diagonal ignores the dense off-diagonal terms the race likelihood genuinely has, and `own_derivative` is used as a proxy for J_{ii} to keep the cost at nothing extra. §16 asks for the full race Hessian to be investigated for understanding; it was not. What was done is to expose `curvature="unit"` so a first-order booster is available as the reference, and the effective-step problem in §10 is a direct consequence of the diagonal’s σ scaling, so the approximation is not innocent.

The comparison is against the raw-softmax measurement of the incumbent, not the calibrated production model. The stage’s comparator is the softmax link on the same margins, which is the right paired control for an uncalibrated candidate. The calibrated production figure is -0.004647 against the -0.003957 used here, and a reader comparing this document’s numbers with the published production report must not mix the two.

Depth was not swept at canonical scale. §35 asks for softmax against probit at matched depth as the direct test of the motivating hypothesis. It is reported at development scale only, because a canonical depth sweep is six times the canonical cost and the plan is explicit that canonical compute goes to a small number of serious candidates.

Betting and simulation artifacts were not produced. The published probit reports carry a bet ledger and bankroll paths from the settlement stage. Those are secondary diagnostics — returns cannot select a model, and this repository says so — and they are absent here rather than reported as empty.

18 Recommendation

Promote to production candidate

Nothing in this category.

Promising but requires another canonical confirmation

Nothing in this category.

Probability improvement but computationally impractical

- *Not this, and it is worth saying why.* Training is about $30\times$ the softmax booster’s and scoring is free, so a weekly rebuild is unaffected and only an interactive full-history refit would hurt. Cost is not what decides this experiment.

No meaningful difference from post-hoc probit

- *Not this.* The boosted margins differ from the Offset margins by -0.000610 scored through the identical softmax link, so the mean function genuinely differs rather than being a rescaling of the incumbent’s.

Negative — keep the softmax booster

- Boosted probit against the production softmax booster: -0.001352 , $t = -1.40$ across 11 test years, 6/11 favourable, 34.2% of the production edge. Against the link-only probit: -0.000410 at $t = -0.46$. Neither margin survives a year-level test. The direction is favourable and the

per-race and per-seed spreads are tight, but the year-to-year spread is the one that speaks to a season this model has not seen, and against it neither comparison clears $|t| > 2$. On the repository’s own standing evidence — eighteen candidates carried to canonical scale in the companion research programme, none confirming — a 6/11 year split is what nothing looks like.

§47’s success criteria, answered

1. **Can the existing probability machinery supply accurate gradients?** **Yes** — bit-identical probabilities, 5.6e-9 gradient agreement.
2. **Can the existing anchor be reused without sacrificing correctness?** **Yes** — unchanged, and exact to 1.1e-16 at zero correction.
3. **Is direct probit training computationally feasible?** **Yes** — two minutes per 400-round fit over a decade of races.
4. **Does the probit objective learn different structure?** **Yes**, measurably, and not as a rescaling.
5. **Does it outperform the existing probit?** See §18 and the ladder: **−0.000410**.
6. **Does it outperform the market-offset probit?** Same comparison.
7. **Does it outperform the softmax booster?** **−0.001352**.
8. **Does any gain survive canonical testing?** That is what §11 reports; the development-scale numbers in this document are labelled as such throughout.
9. **Is any gain large enough to justify the complexity?** Answered in the category above rather than asserted here.